

Audit Proposal

This proposal covers the scope of the audit, provides a quick orientation, and introduces the workflow used to build/test our code.

The code is available at <https://github.com/wolvesofwallstreet/wolves.finance>.

Scope

The audit will cover our ERC-1155 token contracts. The primary contracts are:

- WOWSErc1155.sol
- WOWSCryptofolio.sol

We deployed two logic contracts which are not part of this first audit:

- WOWSSftMinter.sol
- RewardHandler.sol

Our unit tests also depend on two test contracts. These are not deployed on mainnet and don't require auditing, but they serve as a good reference for how the cryptofolios will be used:

- TradeFloor.sol
- TestStakingContact.sol

Orientation

WOWSErc1155.sol is our ERC-1155 token. We don't make use of fungible tokens, so all tokens are NFTs. Our tokens act as cryptofolios, and can contain other NFTs. NFT tokens minted in here are meant to not be tradeable on NFT marketplaces.

WOWSCryptofolio.sol is our cryptofolio "template". This contract is deployed once on mainnet, and when an ERC-1155 token is minted, the contract is cloned. This address of the cloned contract is what receives NFTs being added to the cryptofolio.

WOWSSftMinter.sol is our NFT crowdsale. We've created a simple purchase dapp where users can buy our first round of NFTs. A user can mint an NFT and pay using our ERC-20 token.

RewardHandler.sol is responsible for distributing funds according to our tokenomics.

TradeFloor.sol will be our NFT contract which manages 2 kinds of NFT's:

1.) A snapshot NFT of WOWSErc1155 token which can be traded on NFT marketplaces. Metadata and a composited image will be generated showing the Cryptofolio items.

2.) Cryptofolio Item NFT's (like a "mobile phone", or an investment NFT)

Cryptofolio items can be minted to the address of the WOWSCryptofolio.sol clone.

Workflow (user view)

- User selects an NFT stock card in our dapp. Each stock card in the UI has a level and a cardId. A base tokenId is calculated by: $\text{tokenIdBase} = (\text{level} \ll 24) \mid (\text{cardId} \ll 16)$. Hexadecimal: 0xLLCC0000) where L is level and C is CardId. The lower 16Bit are used for quantities. This means we can manage 256 levels, 256 cardIds per level and 65536 quantities per cardId.
- User calls the SFTMinter contract which: a.) requests the next unminted tokenId from the 65536 possible for a given level and cardId, and b.) mints this token. Both are function calls int ERC1155 (getNextWowsTokenId() / mint())

When a token is minted for the first time, a clone of WOWSCryptofolio is created and 1:1 referenced to a tokenId. This WOWSCryptofolio will later store information about CryptofolioItems owned by itself.

-> User owns an NFT from the WOWSErc1155 contract, the NFT token references to its unique Cryptofolio contract clone.

- User visits our dapp site "Upgrade Cryptofolio".
- There they select one of their WOWSERC1155 NFT as the recipient which should be upgraded.
- This page will also list NFT Cryptofolio items (managed in TradingFloor.sol), which can be used to upgrade the WowsERC1155 token.
- User selects one or more CryptofolioItems.
- User selects "Buy". Our contract(s) will mint NFT token in TradingFloor.sol, recipient (==owner) is the Cryptofolio clone contract which is 1:1 linked to the WowsERC1155 token.

Note: ERC1155 contracts fire an onTokenReceived call to the recipient of ERC1155 tokens on every transfer (including mint). This is done to make sure that the receiver of the ERC1155 token can handle them.

Our implementation of onTokenReceived in WowsCryptofolio.sol records the sending contract and the NFT's which are sent to it before it processes the standard procedure.

The only reason to record this data is to get them queried fast in the future UI workflow. The only alternative would be parsing ETH logs.

-> User owns a WowsERC1155 NFT token which has Cryptofolio items backed.

Next steps just to make workflow transparent:

- User want to trade their WOWSERC1155 on OpenSea. This is not allowed (setApprovalForAll restricted), because the Cryptofolio items could be modified while the WOWSERC1155 is in auction.
- We implement a locking contract which locks (ownership transfer) the WowsERC1155 token and returns a TradingFloor NFT. This TradingFloor NFT gets its own new metadata based on the Cryptofolio items and gets its own composited image / video.
- User can trade this TradingFloor NFT on OpenSea / others.
- The TradingFloor NFT is always exchangeable with the original WOWSERC1155 NFT.

Workflow (unit test view)

Our build system is based on Yarn, with some additional tooling for dependencies. See the main README for available build system steps.

Our yarn steps are typical, except for the command `yarn depends`. This step invokes a custom build system that downloads dependencies for testing (currently WETH and Uniswap), patches some code, runs all test cases to verify correctness of the patches, and then copies the Solidity files to our contracts directory.

Test cases for dependencies can take 5 minutes or so, so there is an option to disable these tests using `yarn depends TEST=0`, which only takes several seconds.

The custom tooling expects standard Linux tools like bash and make. These can be installed on Windows, but it might be easier to build in a Linux VM.

Our own unit tests are executed via `yarn test-chain`. We have coverage of the four contracts listed above.

Notes / Setup

- 1.) WOWSERC1155 contract has initially only one address as admin.
On mainnet / rinkeby these are our Gnosis Multi-Sig wallets.

Before you can mint a WOWSErc1155 token, MINTER_ROLE has to be granted.

- 2.) CryptofolioItems are owned by WowsCryptofolio.sol. To be able to get Cryptofolio items transferred out of this contract, the owner of the “parent” WOWSErc1155 contract (which is generally the user who owns it) gets approved to move CryptofolioItems out of WowsCryptofolio.sol.
Pls. refer to the unit tests to see the workflow.